

# 실전반 2회 · 노이즈를 구별하는 수집 블록

- [이번 회차에서 할 일](#)
- [1. 시작 전 준비 \(복습·환경 확인\)](#)
- [2. 핵심 개념](#)
- [3. 활동 1 — 결측·스파이크·고착 사례 비교 · 50분](#)
- [4. 활동 2 — 품질 기준과 1초 집계 수정 · 50분](#)
- [5. 활동 3 — 원시 값·정제 값·품질 플래그 저장 · 50분](#)
- [6. 활동 4 — 설비 이상과 센서 오류를 다른 상태로 표시 · 50분](#)
- [7. 실습 과제](#)
- [8. 완료 확인 체크리스트](#)
- [9. 회고 \(10분\)](#)
- [강사 노트](#)
- [용어](#)

| 날짜·시간 | 2026-11-07 (토) 14:00~18:00 | 모듈  | M1 · 원 일정 4번 |
|-------|----------------------------|-----|--------------|
| 블록    | 확장 B2 · B3                 | 산출물 | 품질 검사와 이력 저장 |
| 완료 확인 | 긴 결측을 정상 값으로 숨기지 않고 판정 보류  |     |              |

## 회차별 블록 로드맵 — 하나의 시스템에 매 회차 기능을 더한다

**구현** 새로 만든다 · **확장** 조건·연결을 바꾼다 · **플랫폼** 교육용 플랫폼에 연결한다

실전반 4학년 · 9회 × 4시간 (합수·질의·정책 수정과 검증)

|             | B1 | B2 | B3  | B4   | B5 | B6   | B7   | B8   | B9   |
|-------------|----|----|-----|------|----|------|------|------|------|
| 1회<br>10/31 | 구현 | 구현 | 구현  |      |    |      |      |      |      |
| 2회<br>11/07 |    | 확장 | 확장  |      |    |      |      |      |      |
| 3회<br>11/21 |    |    | 확장  | 구현   |    |      |      |      |      |
| 4회<br>11/28 |    | 확장 | 확장  |      | 구현 |      |      | 구현   |      |
| 5회<br>01/25 |    |    |     | 확장   |    | 구현   | 구현   |      |      |
| 6회<br>01/26 |    |    | 플랫폼 | 확장+P |    |      | 확장   |      |      |
| 7회<br>01/27 |    |    |     |      |    | 확장+P | 확장+P | 구현+P |      |
| 8회<br>01/28 | 확장 |    |     |      | 확장 |      |      | 확장   |      |
| 9회<br>01/29 |    |    |     |      |    |      |      |      | 확장+P |

통합반 2·3학년 · 10회 × 4시간 (제공 코드의 매핑·설정·조건 완성)

|              | B1 | B2 | B3  | B4   | B5 | B6   | B7   | B8   | B9   |
|--------------|----|----|-----|------|----|------|------|------|------|
| 1회<br>11/14  | 구현 | 구현 |     |      |    |      |      |      |      |
| 2회<br>01/25  |    | 확장 | 구현  |      |    |      |      |      |      |
| 3회<br>01/26  |    |    |     | 구현   |    |      |      |      |      |
| 4회<br>01/27  |    | 확장 | 확장  |      | 구현 |      |      | 구현   |      |
| 5회<br>01/28  |    |    |     | 확장   |    | 구현   |      |      |      |
| 6회<br>01/29  |    |    |     |      |    |      | 구현   | 구현   |      |
| 7회<br>02/01  |    |    | 플랫폼 | 확장+P |    |      |      |      |      |
| 8회<br>02/02  |    |    |     |      |    | 확장+P | 확장+P | 확장+P |      |
| 9회<br>02/03  |    |    |     |      |    |      |      |      | 확장+P |
| 10회<br>02/04 | 확장 |    |     |      | 확장 |      |      | 확장   | 확장   |

이번 회차가 다루는 블록

## 이번 회차에서 할 일

1회차에서 만든 B1→B2→B3 경로는 제품 품질 검사기를 그대로 썼다. 오늘은 그 검사기의 내부 조건을 직접 고친다. 센서가 값을 보내지 않거나(결측), 한 번 튀거나(급등), 같은 값에 멈추는(고착) 일은 설비가 고장 난 것과 다르다. 이 차이를 구별하지 못하면 뒤 회차의 감지(B5)가 센서 배선 문제에 부하 감소를 제안하거나, 반대로 실제 냉각 저하를 "급등"으로 지워 버린다. 회차가 끝나면 오류 주입본과 시뮬레이터 시나리오에서 결측·급등·고착을 서로 다른 플래그로 나누고, 원시 값·정제 값·플래그를 함께 저장하고, 구간을 "센서 오류 / 설비 이상 / 정상"으로 표시할 수 있다.

### 학습 목표

- 오류 주입본(사이클 #1500)의 결측·급등·고착을 플래그 분포와 위치로 설명한다.
- `QualityRules` 기준(`gap_hold_s=5`, `stuck_run=8`, `spike_delta TS1=8°C`)에 맞게 결측·고착·급등 조건과 1초 집계를 고치고 테스트로 검증한다.
- 원시 값(`raw_value`)을 보존하고 정제 값(`value`)만 비우는 저장 규칙을 SQL 로 확인한다.
- 센서 오류 구간에서 설비 이상 판정을 보류하는 순서를 코드로 구현한다.

## 1. 시작 전 준비 (복습·환경 확인)

```
cd lecture/system
PYTHONPATH=. .venv/bin/python -m pytest ../labs/practical/S01 -q
```

1회차 과제를 마쳤다면 `3 passed` 가 나온다. 품질 기준값을 확인한다.

```
PYTHONPATH=. .venv/bin/python -c "from hydops.config import QR; print(QR)"
```

```
QualityRules(physical_min={'TS1': -10.0, 'PS1': 0.0, 'FS1': 0.0}, physical_max={'TS1': 120.0, 'PS1': 250.0, 'FS1': 30.0}, spike_delta={'TS1': 8.0, 'PS1': 60.0, 'FS1': 15.0}, spike_window=5, stuck_run=8, gap_hold_s=5)
```

**주의** — 모든 기준값은 교육용 가정값이다. UCI 설비나 실제 유압설비의 운전 기준이 아니다.

급등 기준 `spike_delta` 는 정상 동작의 실측에서 정했다. 설정 파일의 주석이 그 근거다.

```
# lecture/system/hydops/config.py (발췌)
# UCI 정상 사이클의 1초 최대 변화: TS1 0.23°C · PS1 40.3 bar · FS1 10.9 L/min (펌프 기동)
→ 그보다 큰 점프만 급등
spike_delta: dict = field(default_factory=lambda: {"TS1": 8.0, "PS1": 60.0,
"FS1": 15.0})
```

1회차에서 축약본 2,205 사이클의 사이클 안 1초 최대 변화를 직접 계산해 {'TS1': 0.23, 'PS1': 40.26, 'FS1': 10.94} 를 확인했다. 초기 설정의 FS1 기준 6 L/min 은 이 정상 변화(0 → 8.94 L/min 펌프 기동)보다 작아서 정상 사이클에서 SPIKE 가 났고, 실측을 근거로 15.0 으로 고쳤다. 오늘 고치는 규칙도 같은 원칙을 따른다 — **기준은 정상 동작을 측정한 뒤 그 바깥에 둔다.**

## 2. 핵심 개념

### 2.1 원시 값과 정제 값

hydops/b2\_quality/checks.py 의 원칙은 파일 첫머리에 적혀 있다.

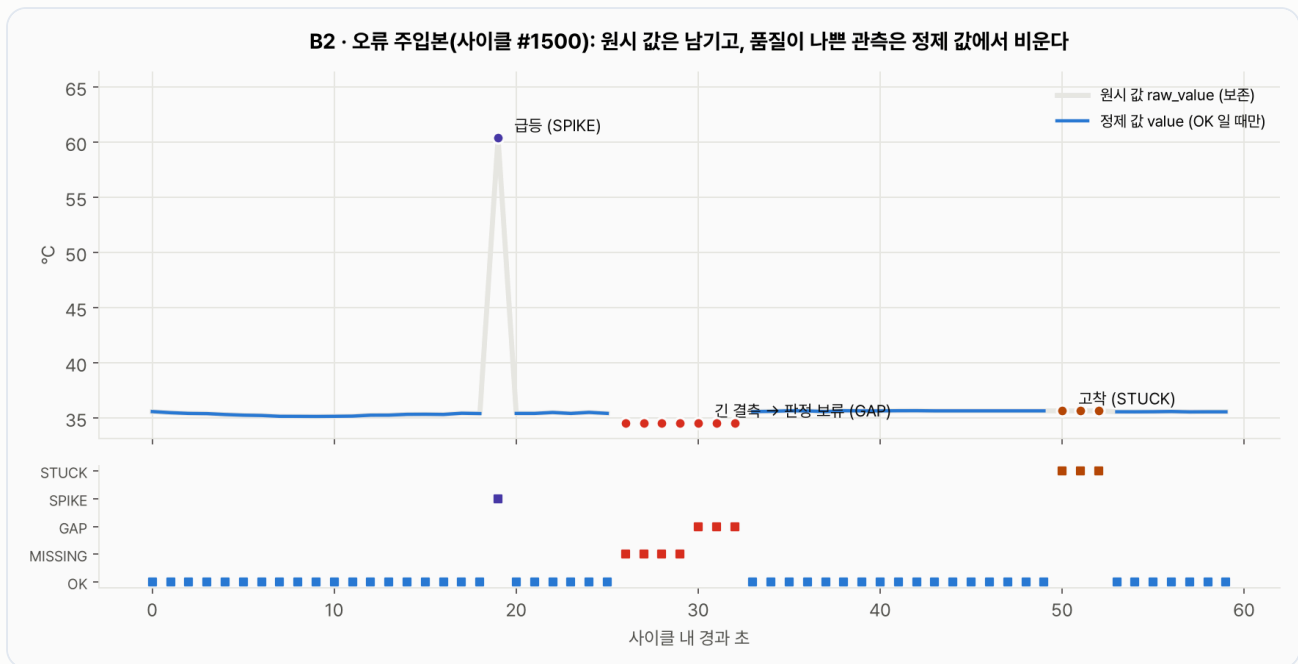
```
# lecture/system/hydops/b2_quality/checks.py (발췌)
"""B2 · 센서 품질 검사.

센서 오류(결측·급등·고착·범위 이탈)와 설비 이상을 구분하기 위해 관측마다 품질 플래그를 붙인다.
원시 값(raw_value)은 그대로 보존하고, 정제 값(value)은 품질이 OK 일 때만 채운다.
긴 결측을 보간해서 정상 값처럼 숨기지 않는다.
"""
OK, MISSING, GAP, SPIKE, STUCK, OUT_OF_RANGE = "OK", "MISSING", "GAP", "SPIKE",
"STUCK", "OUT_OF_RANGE"
```

| 플래그     | 조건 (TS1 기준)   | 정제 값 | 뒤 블록의 해석                |
|---------|---------------|------|-------------------------|
| OK      | 아래 어느 것도 아님   | 원시 값 | 설비 판단에 쓴다               |
| MISSING | 결측 1~4초째      | NULL | 짧은 결측, 지속 카운터 유지        |
| GAP     | 결측이 5초째 이상    | NULL | 판정 보류 →<br>SENSOR_FAULT |
| STUCK   | 같은 값 8번째 반복부터 | NULL | 판정 보류 →<br>SENSOR_FAULT |

| 플래그          | 조건 (TS1 기준)                          | 정제 값 | 뒤 블록의 해석  |
|--------------|--------------------------------------|------|-----------|
| SPIKE        | 직전 유효값과 1초 사이 8°C 초과 점프 (3회 연속이면 OK) | NULL | 단발은 기록만   |
| OUT_OF_RANGE | -10°C 미만 또는 120°C 초과                 | NULL | 물리적으로 불가능 |

## 2.2 오류 주입본



오류 주입본의 플래그

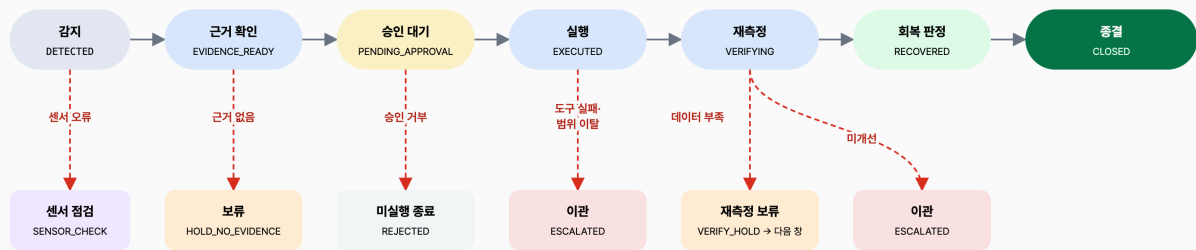
사이클 #1500 에 `uci.inject_sensor_errors` 로 급등 1건(+25°C), 결측 7초, 고착 10초를 넣은 결과다. 회색 굵은 선이 원시 값, 파란 선이 정제 값이다. 결측 7초 중 앞 4초는 MISSING, 5초째부터 GAP 이다. 고착 10초 중 STUCK 은 3초뿐인데, 같은 값이 8번째 나올 때 비로소 "고착"이라 판정하는 스트리밍 방식이라 앞 7초는 OK 로 지나가기 때문이다.

**주의** — 고착 앞 7초가 OK 로 저장되는 것은 버그가 아니라 스트리밍 판정의 지연이다. 나중에 되돌아가 앞 7초를 고쳐 쓰지 않는다. 대신 구간 판정( `classify_window` )이 STUCK 이 하나라도 있으면 구간 전체를 SENSOR\_FAULT 로 본다. 이 선택이 맞는지는 회고에서 토론한다.

## 2.3 사건 상태 흐름에서의 위치

## 사건 상태 흐름 — 정상 경로와 분기

명령 성공(action\_log.command\_status)과 회복 성공(verification.outcome)은 별도 필드다. 재시작 후 같은 사건을 다시 읽어도 이미 실행한 조치를 반복하지 않는다.



코드가 지키는 합격 목표: 근거 없는 조치 0건 · 승인 없는 실행 0건 · 중복 실행 0건 · 재확인 없는 회복 판정 0건 (LLM 이 만든 approved=true 는 승인으로 인정하지 않는다)

## 사건 상태 흐름

맨 왼쪽 "감지 → 센서 점검(SENSOR\_CHECK)" 분기가 오늘 만드는 품질 플래그에서 시작한다. B5 감지기는 GAP·STUCK 을 보면 COOLING\_ANOMALY 가 아니라 SENSOR\_FAULT 사건을 만들고, 설비 조치를 보류한다.

## 3. 활동 1 — 결측·스파이크·고착 사례 비교 · 50분

### 목표

오류 주입본과 시뮬레이터 센서 고장 시나리오에서 세 가지 오류가 어떤 플래그로, 몇 개, 어디에 생기는지 비교한다.

### 따라 하기

1. 제공 검사기로 오류 주입본을 검사한다.

```

PYTHONPATH=. .venv/bin/python -c "
from datetime import datetime, timezone
from hydops.b1_data import uci
from hydops.b2_quality.checks import SensorQualityChecker, window_quality,
    classify_window
ds = uci.load_reduced()
rows = uci.inject_sensor_errors(uci.cycle_observations(ds, 1500, 'HYD-01',
    datetime(2026, 11, 7, tzinfo=timezone.utc)))
checked = [r for r in SensorQualityChecker().check_many(rows) if
    r['sensor_id'] == 'TS1']
print(window_quality(checked), classify_window(checked))
  
```

```
print([(r['elapsed_s'], r['raw_value'], r['quality_flag']) for r in checked
       if r['quality_flag'] != 'OK'])
"
```

```
{'n': 60, 'valid': 49, 'valid_ratio': 0.817, 'flags': {'OK': 49, 'SPIKE': 1, 'MISSING': 4, 'GAP': 3, 'STUCK': 3}, 'longest_gap_s': 7} SENSOR_FAULT
[(19, 60.414, 'SPIKE'), (26, None, 'MISSING'), (27, None, 'MISSING'), (28, None, 'MISSING'), (29, None, 'MISSING'), (30, None, 'GAP'), (31, None, 'GAP'), (32, None, 'GAP'), (50, 35.652, 'STUCK'), (51, 35.652, 'STUCK'), (52, 35.652, 'STUCK')]
```

19초의 급등값 60.414°C 는 경보 온도 60°C 를 넘는다. 그러나 1초뿐이므로 사건이 되면 안 된다.

2. 시뮬레이터 시나리오를 비교한다. 냉각 성능을 0.25 로 떨어뜨린 뒤(심각한 저하) 센서 고장을 15 초 주입하고, 품질 검사기와 B5 감지기 `ThresholdDetector` 에 흘린다. DB 에 쓰지 않는 순수 계산이다.

```
# 실행 스크립트 발췌 (시드 7, 재생 시작 2026-09-02 00:00 UTC)
sim = HydraulicSimulator("HYD-01", seed=7, start=T0)
r = sim.run(20); sim.inject_cooling_degradation(0.25); r += sim.run(60)
if fault:
    sim.inject_sensor_fault(fault, 15); r += sim.run(20)
ck = SensorQualityChecker().check_many(r)
det = ThresholdDetector()
for row in ck:
    for s in det.feed(row):
        sigs.append((s.event_type, s.window_end.strftime("%H:%M:%S"),
                     s.evidence.get("flag")))
```

| 주입          | TS1 플래그 분포<br>(100초 또는 80초)      | 마지막 60초<br><code>classify_window</code> | 감지 신호                                                         |
|-------------|----------------------------------|-----------------------------------------|---------------------------------------------------------------|
| 없음          | OK 80                            | VALID                                   | COOLING_ANOMALY 00:00:34                                      |
| stuck 15초   | OK 92 ·<br>STUCK 8               | SENSOR_FAULT                            | COOLING_ANOMALY 00:00:34,<br>SENSOR_FAULT 00:01:28<br>(STUCK) |
| dropout 15초 | OK 85 ·<br>MISSING 4 ·<br>GAP 11 | SENSOR_FAULT                            | COOLING_ANOMALY 00:00:34,<br>SENSOR_FAULT 00:01:25 (GAP)      |



그래프의 빨간 점이 결측 구간이고, 배지는 OK 58 · MISSING 4 · GAP 21 이다. 사건은 "센서 오류 HYD-01", 감지 근거는 flag GAP · rule quality-gap-stuck-v1 · gap\_hold\_s 5 · stuck\_run 8 이고, 결과는 SENSOR\_CHECK 다.

## 확인 포인트

- 결측은 MISSING 4 + GAP 3 으로 나뉘고, 최장 결측 longest\_gap\_s 는 7이다.
- 단발 급등과 spike 주입은 구간을 SENSOR\_FAULT 로 만들지 않는다. 결측·고착은 만든다.

## 흔한 실수

- GAP 을 "설비 정지"로 해석한다. 같은 시각 PS1-FS1 이 정상이면 SOP-SEN-001 §3 이 말하듯 센서 오류일 가능성이 높다.
- 급등 1건만 보고 SENSOR\_FAULT 사건을 만든다. SOP-SEN-001 §2 는 "단발 급등은 한 번이면 기록만 하고 발동하지 않는다"고 적는다.

# 4. 활동 2 — 품질 기준과 1초 집계 수정 · 50분

## 목표

labs/practical/S02/quality.py 의 aggregate\_1s 와 LabQualityChecker.check 를 기준에 맞게 고친다.

## 따라 하기

이 활동의 비교 출력은 시작본( st )과 정답본( sol )을 함께 불러와 실행한 것이다.

```
import importlib.util
def load(path, name):
    spec = importlib.util.spec_from_file_location(name, path)
    mod = importlib.util.module_from_spec(spec); spec.loader.exec_module(mod);
    return mod
st = load("../labs/practical/S02/quality.py", "st") # 시작본 (내가 고치는 파일)
sol = load("../labs/practical/S02/solution/quality.py", "sol") # 정답본
```

1. 1초 집계. 고주파 센서는 1초 안의 일부 샘플만 빠질 수 있다. 시작본은 결측을 0 으로 채운다.

```
# lecture/labs/practical/S02/quality.py (시작본 발췌)
for sec in shaped:
    # TODO(학생): 결측 샘플(NaN)을 0 으로 채우고 n 을 hz 로 고정한다. ...
    filled = np.nan_to_num(sec, nan=0.0)
    out.append({'mean': float(filled.mean()), 'min': float(filled.min()),
               "max": float(filled.max()), "n": hz})
```

사이클 #100 PS1 의 6초째 100개 샘플 중 앞 70개를 NaN 으로 만들고( `smp[600:670] = np.nan` ) 비교한 결과다. 이 절의 출력 앞머리 `starter / solution` 은 실행한 모듈(시작본/정답본)을 표시하고, `full mean sec6` 은 남은 30개 샘플( `smp[670:700]` )의 평균이다.

```
starter   PS1 sec6 (70 NaN): {'mean': 56.9979, 'min': 0.0, 'max': 190.23,
'n': 100} full mean sec6 189.99300000000002
solution  PS1 sec6 (70 NaN): {'mean': None, 'min': None, 'max': None, 'n':
30} full mean sec6 189.99300000000002
```

남은 30개 샘플의 평균은 189.99 bar 인데, 0 으로 채운 평균은 57.00 bar 로 "압력이 급락한 것"처럼 보인다. 정답본은 유효 샘플만 세고( `n=30` ), 유효가  $hz \times 0.5$  미만이면 그 1초를 결측( `mean=None` )으로 넘겨 품질 검사가 MISSING/GAP 으로 판정하게 한다.

2. 긴 결측. 시작본은 결측을 항상 MISSING 으로 두고 정제 값을 직전 유효값으로 채운다.

```
# 시작본
if raw is None:
    self._missing_run[key] += 1
    # TODO(학생): 결측이 아무리 길어도 MISSING 이고, 정제 값을 직전 유효값으로 채운다
    (forward fill). ...
    flag = MISSING
    self._same_run[key] = 0
    recent = self._recent[key]
    return {**row, "quality_flag": flag, "value": recent[-1] if recent else
           None}
```

오류 주입본 24~33초를 시작본과 정답본으로 검사한 결과 ( `(elapsed_s, raw_value, flag, value)` ):

```
starter   [(24, 35.516, 'OK', 35.52), (25, 35.426, 'OK', 35.43), (26, None,
'MISSING', 35.43), (27, None, 'MISSING', 35.43), (28, None, 'MISSING',
35.43), (29, None, 'MISSING', 35.43), (30, None, 'MISSING', 35.43), (31,
None, 'MISSING', 35.43), (32, None, 'MISSING', 35.43), (33, 35.586, 'OK',
35.59)]
solution  [(24, 35.516, 'OK', 35.52), (25, 35.426, 'OK', 35.43), (26, None,
'MISSING', None), (27, None, 'MISSING', None), (28, None, 'MISSING', None),
```

```
(29, None, 'MISSING', None), (30, None, 'GAP', None), (31, None, 'GAP', None), (32, None, 'GAP', None), (33, 35.586, 'OK', 35.59)]
```

시작본은 7초 결측 동안 35.43°C 가 이어진 것처럼 정제 값을 채운다. 긴 결측을 정상 값으로 숨긴 것이다. 정답본은 `gap_hold_s` 이상에서 GAP 으로 바꾸고 정제 값을 비운다.

3. 고착. 시작본은 `self._same_run[key] > self.rules.stuck_run` 이라 9번째부터 STUCK 이다. 10초 고착에서 STUCK 이 2개로 줄고 테스트가 `assert 2 == 3` 으로 실패한다. `>=` 로 고친다.
4. 급등과 수준 변화. 시작본은 최근 유효값들의 중앙값과 비교하고, 연속 점프를 수준 변화로 받아들이지 않는다.

```
# 시작본
elif recent and abs(raw - statistics.median(recent)) >
    self.rules.spike_delta.get(sid, float("inf")):
    self._spike_run[key] += 1
    flag = SPIKE
```

정답본은 완성 시스템과 같다.

```
# lecture/system/hydops/b2_quality/checks.py (발췌) = S02 정답본
elif recent and abs(raw - recent[-1]) > self.rules.spike_delta.get(sid,
    float("inf")):
    # 1초 사이에 물리적으로 불가능한 점프 → 급등. 단, 3회 이상 연속이면 수준 변화로 받아들인다
    self._spike_run[key] += 1
    flag = SPIKE if self._spike_run[key] < 3 else OK
```

냉각 성능 0.25(시드 7) 시나리오 80초를 두 검사기로 돌린 결과다.

```
starter   severe 0.25 flags: {'OK': 22, 'SPIKE': 58}
solution  severe 0.25 flags: {'OK': 80}
```

이 시나리오에서 1초 사이 최대 상승은 3.57°C 로 기준 8°C 보다 작다. 그런데 중앙값은 최근 몇 초 전 값이라 빠르게 오르는 온도보다 뒤쳐지고, 한 번 SPIKE 로 막힌 값은 `recent` 에 들어가지 않아 기준이 계속 낡는다. 결국 58개가 SPIKE 로 막힌다. 정제 값이 비면 B5 감지기는 유효 관측을 세지 못하므로 실제 냉각 이상 사건이 생기지 않는다. 완성 시스템을 만들 때 실제로 겪은 문제다.

## 확인 포인트

- 오류 주입본 분포가 정확히 {'OK': 49, 'SPIKE': 1, 'MISSING': 4, 'GAP': 3, 'STUCK': 3} 다.
- 45°C 대에서 60°C 대로 계단 변화하면 처음 두 값만 SPIKE, 세 번째부터 OK 다.
- 냉각 심각 저하의 빠른 상승에서 SPIKE 가 0 이다.

## 흔한 실수

- 급등 기준을 근거 없이 크게 올려(예: TS1 30°C) SPIKE 문제를 덮는다. FS1 을 15.0 으로 올린 것은 정상 사이클 실측 최대 10.94 L/min 이라는 근거가 있지만, TS1 의 정상 1초 최대 변화는 0.23°C 이므로 30°C 로 올릴 근거가 없다. 그러면 +30°C 에 노이즈가 더해지는 시뮬레이터 spike 주입도 기준에 따라 통과할 수 있고, 오류 주입본의 +25°C 급등은 확실히 통과한다. 비교 대상(직전 유효값)과 수준 변화 규칙을 고치는 것이 맞다.
- 1초 집계에서 NaN 이 섞인 배열에 `np.mean` 을 그대로 써 `nan` 을 만든다. `nan` 은 `None` 이 아니므로 품질 검사가 결측으로 보지 못한다.

## 5. 활동 3 — 원시 값·정제 값·품질 플래그 저장 · 50분

### 목표

검사 결과를 `LAB-` run 으로 저장하고, 원시 값은 보존되고 정제 값만 비었는지 SQL 로 확인한다.

### 따라 하기

1. 시작본 `store_checked` 는 정제 값으로 원시 값을 덮어쓴다.

```
# 시작본
# TODO (학생): 정제 값만 남기려고 원시 값을 덮어쓴다. ...
rows = [{**r, "raw_value": r.get("value")} for r in rows]
return store.insert_observations(run_id, rows)
```

이렇게 저장하면 SPIKE·STUCK 행의 원시 값까지 NULL 이 되어, 나중에 "그때 센서가 무슨 값을 보냈는지" 다시 볼 수 없다. 덮어쓰기 줄을 지운다.

2. 정답본으로 오류 주입본을 저장한다.

```
run_id = "LAB-P02-demo"
rows = uci.inject_sensor_errors(uci.cycle_observations(ds, 1500, "HYD-01",
T0))
sol.store_checked(run_id, sol.LabQualityChecker().check_many(rows))
```

3. 품질이 OK 가 아닌 행을 조회한다.

```
docker compose exec -T postgres psql -U hydops -d hydops -c "SELECT
elapsed_s, raw_value, value, quality_flag FROM observation WHERE
run_id='LAB-P02-demo' AND sensor_id='TS1' AND quality_flag <> 'OK'
ORDER BY ts"
```

| elapsed_s | raw_value | value | quality_flag |
|-----------|-----------|-------|--------------|
| 19        | 60.414    |       | SPIKE        |
| 26        |           |       | MISSING      |
| 27        |           |       | MISSING      |
| 28        |           |       | MISSING      |
| 29        |           |       | MISSING      |
| 30        |           |       | GAP          |
| 31        |           |       | GAP          |
| 32        |           |       | GAP          |
| 50        | 35.652    |       | STUCK        |
| 51        | 35.652    |       | STUCK        |
| 52        | 35.652    |       | STUCK        |

(11 rows)

4. 플래그별로 원시 값과 정제 값이 몇 개 남았는지 센다.

```
SELECT quality_flag, count(*) AS n, count(raw_value) AS raw_kept,
count(value) AS clean
FROM observation WHERE run_id='LAB-P02-demo' AND sensor_id='TS1' GROUP BY 1
ORDER BY 1;
```

| quality_flag | n  | raw_kept | clean |
|--------------|----|----------|-------|
| GAP          | 3  | 0        | 0     |
| MISSING      | 4  | 0        | 0     |
| OK           | 49 | 49       | 49    |
| SPIKE        | 1  | 1        | 0     |
| STUCK        | 3  | 3        | 0     |

(5 rows)

5. 자기 행을 지운다: `sol.delete_lab_run("LAB-P02-demo")`.

## 확인 포인트

- 원시 값이 빈 행은 결국 7행(MISSING 4 + GAP 3)뿐이다.
- 정제 값이 빈 행은 11행(품질 불량 전체)이다.
- 1회차에서 본 `agg` 와 오늘의 `raw_value` · `value` · `quality_flag` 가 한 행에 함께 있어, 이력만으로 품질 판정을 다시 검토할 수 있다.

## 흔한 실수

- 결측을 행 자체를 넣지 않는 방식으로 처리한다. 그러면 "7초 결측"이 DB 에서 "행이 없는 7초"가 되어 `window_quality` 가 결측 길이를 셀 수 없다. 결측도 `raw_value NULL` 인 행으로 남긴다.
- 품질 검사를 DB 적재 뒤 SQL 로 한다. 스트리밍 검사기는 센서별 상태(직전 유효값, 같은 값 연속 수)를 기억해야 하므로 적재 전에 한 번 돌린다.

## 6. 활동 4 — 설비 이상과 센서 오류를 다른 상태로 표시 · 50분

### 목표

`window_state` 를 고쳐 TS1 구간을 SENSOR\_FAULT / EQUIPMENT\_ANOMALY / NORMAL 로 나누고, 센서를 믿을 수 없을 때 설비 판정을 보류한다.

### 따라 하기

1. 완성 시스템의 구간 판정을 읽는다. 센서 오류 조건만 본다.

```
# lecture/system/hydops/b2_quality/checks.py (발췌)
def classify_window(rows: list[dict], rules: QualityRules = QR) -> str:
    """구간이 '센서 오류' 인지 판단한다. 센서 오류면 설비 조치를 보류한다."""
    q = window_quality(rows)
    if q["n"] == 0:
        return "NO_DATA"
    if q["longest_gap_s"] >= rules.gap_hold_s or q["flags"].get(STUCK, 0) > 0:
        return "SENSOR_FAULT"
    if q["valid_ratio"] < 0.7:
        return "SENSOR_FAULT"
    return "VALID"
```

2. 시작본 `window_state` 의 두 가지 잘못을 찾는다.

```
# 시작본
for r in rows:
    # TODO (학생): 원시 값으로 센다. ...
    if r["raw_value"] is not None and r["raw_value"] > alarm_c:
        run += 1
        best = max(best, run)
    else:
        run = 0
# TODO (학생): 설비 이상을 먼저 판정한다. ...
if best >= sustain_s:
    state = EQUIPMENT_ANOMALY
elif q["flags"].get(STUCK, 0) > 0 or q["valid_ratio"] < 0.7:
    state = SENSOR_FAULT
```

3. 경보 온도 위에서 센서가 고착된 시나리오(냉각 0.25 + stuck 15초)의 마지막 60초를 두 함수로 판정한다.

```
starter   severe+stuck: EQUIPMENT_ANOMALY {'SPIKE': 53, 'STUCK': 7} 60
solution severe+stuck: SENSOR_FAULT {'OK': 52, 'STUCK': 8} 52
```

(출력 형식: 상태, 플래그 분포, 경보 온도 초과 최장 연속 초.) 시작본은 고착된 원시 값까지 60초 연속 "60°C 초과"로 세어 설비 이상이라 한다. 정답본은 STUCK 이 있으므로 센서 오류로 보류한다. 정답본에서도 유효 값만으로 52초 연속 초과가 있지만, 센서를 믿을 수 없는 구간에서는 설비 판정을 하지 않는다 — SOP-SEN-001 §6 "센서가 복구되기 전에는 냉각 이상 여부를 판정하지 않는다".

4. 세 경우를 모두 확인한다.

| 입력                          | 정답본 <code>window_state</code>    |
|-----------------------------|----------------------------------|
| 오류 주입본 #1500 TS1 60초        | SENSOR_FAULT                     |
| 시뮬레이터 정상 60초 (시드 7)         | NORMAL                           |
| 냉각 0.25 마지막 60초             | EQUIPMENT_ANOMALY (유효 초과 최장 56초) |
| 냉각 0.25 + stuck 15초 마지막 60초 | SENSOR_FAULT                     |

5. 실습 테스트를 돌린다.

```
PYTHONPATH=., .venv/bin/python -m pytest ../labs/practical/S02 -q
```

## 확인 포인트

- 같은 온도 그래프라도 품질 플래그에 따라 상태가 달라진다.
- 설비 이상(EQUIPMENT\_ANOMALY)은 품질 OK 인 정제 값만으로 센다.

## 흔한 실수

- SENSOR\_FAULT 구간에서 유효 값이 52초 연속 60°C 를 넘으니 "설비 이상도 함께" 표시한다. 두 사건을 동시에 올리면 뒤 회차 에이전트가 센서 점검과 부하 감소를 함께 제안할 수 있다. 오늘의 규칙은 센서 오류 우선 보류다.
- "설비 이상"을 제품 불량이라 부른다. 오늘 다루는 것은 설비 상태이며, 합성 제품 검사값은 4회차에 별도로 다룬다.

## 7. 실습 과제

- 폴더: labs/practical/S02/ — TASK.md , quality.py (시작본), solution/quality.py , test\_check.py
- 수정할 함수( # TODO(학생) 8곳): aggregate\_1s (유효 샘플만 집계), LabQualityChecker.check (GAP·정제 값 비우기, 고착 >= , 직전 유효값 기준 급등 + 3회 연속 수준 변화), store\_checked (원시 값 보존), window\_state (정제 값으로 세기, 센서 오류 우선)
- 검증 방법: 오류 주입본 분포, 10초 고착 STUCK 3개, 단발 급등 1개와 계단 변화 수용, 냉각 0.25에서 SPIKE 0 · EQUIPMENT\_ANOMALY, 고착 시 SENSOR\_FAULT, LAB-S02- run 저장 후 원시 값 NULL 7행·정제 값 NULL 11행.
- 확인 명령:

```
cd lecture/system && PYTHONPATH=., .venv/bin/python -m pytest  
../labs/practical/S02 -q
```

시작본 실행 결과(요약):

```
E AssertionError: assert {'OK': 50, 'S...7, 'STUCK': 2} == {'OK': 49,
'S...'GAP': 3, ...}
E AssertionError: 냉각 심각 저하(0.25)의 빠른 상승은 급등이 아니다
assert 58 == 0
E AssertionError: 결측 7초만 원시 값이 비어 있어야 한다
assert 3 == 7
7 failed, 1 passed in 0.25s
```

모두 고친 뒤: 8 passed .

## 8. 완료 확인 체크리스트

- ☐ 오류 주입본에서 결측 7초가 MISSING 4 · GAP 3 으로 나뉘고, 결측 구간의 정제 값이 모두 NULL 이다.
- ☐ 결측 구간을 직전 유효값으로 채우지 않는다(시작본과 출력 비교를 남겼다).
- ☐ 구간에 GAP 또는 STUCK 이 있으면 `window_state` 가 SENSOR\_FAULT(판정 보류)다.
- ☐ 냉각 심각 저하의 빠른 상승이 SPIKE 로 막히지 않는다.
- ☐ 1초 집계에서 결측 샘플을 0 으로 채우지 않고 유효 샘플 수  $n$  을 남긴다.
- ☐ DB 에서 원시 값 NULL 7행, 정제 값 NULL 11행을 확인했다.
- ☐ `pytest ../labs/practical/S02 -q` 가 8 passed 다.
- ☐ 자기 LAB- 행을 지웠다.

## 9. 회고 (10분)

1. 10초 고착 중 앞 7초는 OK 로 저장된다. 이 7초를 나중에 STUCK 으로 고쳐 쓰는 것과 구간 전체를 SENSOR\_FAULT 로 보는 것 중 무엇이 이력 관점에서 나은가?
2. FS1 급등 기준은 정상 사이클 실측 최대 변화 10.94 L/min 을 근거로 15.0 이다. 새 설비(HYD-03)처럼 정상 운전 데이터가 아직 없는 센서의 급등 기준은 어떻게 정하고, 언제 다시 재깎는가?
3. 긴 결측을 보간해서 그래프를 매끄럽게 보여 주면 운전자에게 어떤 오판이 생기는가?

**개인 설명 과제 — 내가 설명할 질의 또는 실패 분기 하나:** (a) 중앙값 급등 규칙이 냉각 심각 저하에서 사건을 막은 실패 분기, (b) forward fill 이 긴 결측을 숨긴 실패 분기, (c) 플래그별 `count(raw_value) · count(value)` SQL, (d) 고착 시 SENSOR\_FAULT 우선 판정 중 하나를 골라 2분 안에 설명한다.

## 강사 노트

### 시간 배분

| 시각          | 내용                                                          |
|-------------|-------------------------------------------------------------|
| 14:00~14:50 | 활동 1 사례 비교 (F03 그림 해설 15분, 시뮬레이터 표 20분, 관제 화면 결측 주입 시연 15분) |
| 14:50~15:40 | 활동 2 검사기 수정 (급등 규칙은 시작본 58 SPIKE 출력을 먼저 보여 준다)              |
| 15:40~16:10 | 휴식 30분                                                      |
| 16:10~17:00 | 활동 3 저장·SQL                                                 |
| 17:00~17:50 | 활동 4 상태 표시, pytest                                          |
| 17:50~18:00 | 회고                                                          |

### 사전 준비

- DB 두 개가 떠 있는지 확인한다. 오늘은 Neo4j 를 쓰지 않는다.
- 관제 화면 시연용으로 `scripts/servers.sh local 4` 를 강사가 띄우고, 시연 직전에 "센서 결측" 버튼으로 S02 화면과 같은 상황을 만든다. 시연 뒤 사건은 SENSOR\_CHECK 로 닫히므로 다른 감지를 막지 않는다.
- 1회차 S01 을 끝내지 못한 학생은 `solution/mapping.py` 를 참고해 15분 안에 따라잡게 한다.

### 장애 대응

- 수업 서버를 쓸 수 없으면 활동 1 의 관제 화면은 S02 스크린샷으로 설명한다. 활동 1~2·4 는 DB 없이 돌아가는 순수 계산이다.
- 활동 3 에서 DB 연결이 실패하면 `test_store_raw_clean_flag` 만 실패한다. 나머지 7개 테스트로 진행하고 저장 확인은 다음 회차 시작 때 보충한다.

## 용어

| 용어              | 뜻                                 |
|-----------------|-----------------------------------|
| 원시 값(raw_value) | 센서가 보낸 1초 값. 결측이 아니면 품질이 나빠도 보존한다 |

| 용어                 | 뜻                                                              |
|--------------------|----------------------------------------------------------------|
| 정제 값(value)        | 품질 OK 일 때만 채우는 값. 감지는 이 값만 쓴다                                  |
| MISSING / GAP      | 짧은 결측 / gap_hold_s(5초) 이상 이어진 결측. GAP 은 판정 보류 신호               |
| STUCK              | 같은 값이 stuck_run(8)번째 반복된 순간부터의 고착                              |
| SPIKE              | 직전 유효값과 1초 사이 기준 초과 점프. 3회 연속이면 수준 변화로 받아들인다                   |
| 수준 변화(level shift) | 값이 새 수준으로 옮겨가 유지되는 것. 급등과 구분한다                                 |
| 판정 보류              | 센서를 믿을 수 없어 설비 이상 여부를 판단하지 않는 상태 (SENSOR_FAULT → SENSOR_CHECK) |
| forward fill       | 결측을 직전 값으로 채우는 보간. 긴 결측에는 쓰지 않는다                               |
| 오류 주입본             | 사이클 #1500 에 급등 1·결측 7초·고착 10초를 넣은 교육용 데이터                      |